

**“Año de la Esperanza y el Fortalecimiento de la
Democracia”**



UPLA

TEMA:

ACTIVIDAD 02

CURSO: ALGORITMOS Y ESTRUCTURA DE DATOS

DOCENTE: FERNÁNDEZ BEJARANO RAUL ENRIQUE

INTEGRANTES: CONDORI MORENO JOSHEP E.

FACULTAD: INGENIERIA

E.A.P: INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

2026

1. Método para eliminar un elemento "x" de la matriz (se reemplaza por 0)

```
// 1. Método para eliminar un elemento "x" de la matriz (se reemplaza por 0)
public void eliminar(int x) {
    boolean encontrado = false;
    for (int i = 0; i < filas; i++) {
        for (int j = 0; j < columnas; j++) {
            if (matriz[i][j] == x) {
                matriz[i][j] = 0;
                encontrado = true;
                System.out.println("Elemento " + x + " eliminado en la posición " + i + " " + j);
                return;
            }
        }
    }
    if (!encontrado) {
        System.out.println("El elemento " + x + " no se encuentra en la matriz.");
    }
}
```



MÉTODO MATRIZ 1

Eliminar Elemento en Matriz

Busca la primera aparición del valor "x" en la matriz bidimensional y lo elimina reemplazándolo por 0.

**¿Qué hace?**

Recorre la matriz bidimensional por filas y columnas (filas x columnas). Al encontrar la primera coincidencia del número x, cambia su valor por 0, marca encontrado = true e interrumpe inmediatamente la ejecución con return.

Idea principal

"En una matriz fija, 'eliminar' consiste en sobrescribir con 0 la casilla encontrada y salir de inmediato."

**Ejemplo**

Eliminar el valor x = 7 de la matriz:

Antes

3	1	5
4	7	9

→

Después

3	1	5
4	0	9

Se encontró 7 en posición [1][1] y cambió a 0

Paso a paso del algoritmo

1 Inicialización y Recorrido

Inicia encontrado = false. Usa bucles anidados para recorrer fila por fila (i) y columna por columna (j).

```
boolean encontrado = false;
for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        ...
    }
}
```

Doble for: Filas y Columnas

2 Coincidencia y Reemplazo

Si matriz[i][j] == x, asigna 0 a esa posición, marca la bandera como true e informa la eliminación.

```
if (matriz[i][j] == x) {
    matriz[i][j] = 0;
    encontrado = true;
    System.out.println("Elemento...");
    return;
}
```

Reemplazo por 0 + return

3 Validación de Ausencia

Si los ciclos terminan sin encontrar x (no ocurrió el return), verifica !encontrado e imprime el mensaje correspondiente.

```
if (!encontrado) {
    System.out.println(
        "El elemento " + x +
        " no se encuentra..."
    );
}
```

Notificación si no existe

Prueba de Escritorio (Buscar x = 7)

Fila (i)	Col (j)	matriz[i][j]	Evaluación	Acción
0	0	3	3 == 7 (Falso)	Sigue búsqueda
0	1	1	1 == 7 (Falso)	Sigue búsqueda
1	0	4	4 == 7 (Falso)	Sigue búsqueda
1	1	7	7 == 7 (Verdadero)	matriz[1][1]=0; return;

Conceptos Clave del Método

• Salida temprana con return:
Al ejecutar return, el método finaliza inmediatamente. Esto evita seguir recorriendo innecesariamente el resto de la matriz y elimina únicamente la **primera aparición** de x.

• Manejo de estado con boolean:
La variable encontrado sirve como bandera lógica para saber si el elemento existía en la matriz en caso de que no se use un return directo.

En resumen

1. Recorrer con for anidado → 2. ¿matriz[i][j] == x? → 3. Asignar 0 y return → 4. Si no, notificar ausencia

2. Método para modificar un elemento "x" de la matriz por un nuevo valor

```
// 2. Método para modificar un elemento "x" de la matriz por un nuevo valor
public void modificar(int x, int nuevoValor) {
    boolean encontrado = false;
    for (int i = 0; i < filas; i++) {
        for (int j = 0; j < columnas; j++) {
            if (matriz[i][j] == x) {
                matriz[i][j] = nuevoValor;
                encontrado = true;
                System.out.println("Elemento " + x + " modificado por " + nuevoValor);
                return;
            }
        }
    }
    if (!encontrado) {
        System.out.println("El elemento " + x + " no se encuentra en la matriz.");
    }
}
```



MÉTODO MATRIZ 2
Modificar Elemento en Matriz
Busca la primera coincidencia de "x" en la matriz y reemplaza su contenido por "nuevoValor".



**¿Qué hace?**

Recorre la matriz bidimensional mediante ciclos anidados. Cuando ubica la primera aparición de x, actualiza la posición con nuevoValor, cambia la bandera encontrado = true y finaliza la ejecución con return.

**Idea principal**

“Ubicamos el elemento objetivo y sobrescribimos su celda directamente con el nuevo valor deseado.”

**</> Ejemplo**

Modificar x = 7 por nuevoValor = 99:

Antes

3	1	5
4	7	9

Después

3	1	5
4	99	9

Se halló 7 en [1][1] y se actualizó a 99

Paso a paso del algoritmo

1 Inicializar y Recorrer

Inicia encontrado = false. Recorre cada fila (i) y cada columna (j) buscando el valor x.

```
boolean encontrado = false;
for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        ...
    }
}
```

Exploración matricial

2 Actualizar y Retornar

Al coincidir matriz[i][j] == x, asigna nuevoValor, marca la bandera true y detiene el proceso con return.

```
if (matriz[i][j] == x) {
    matriz[i][j] = nuevoValor;
    encontrado = true;
    System.out.println("Elemento...");
    return;
}
```

Asignación directa

3 Reportar Ausencia

Si los bucles finalizan sin encontrar x, evalúa !encontrado para mostrar un mensaje informativo.

```
if (!encontrado) {
    System.out.println(
        "El elemento " + x +
        " no se encuentra..."
    );
}
```

Mensaje de control

Prueba de Escritorio (Modificar x = 7 a 99)

Fila (i)	Col (j)	matriz[i][j]	Evaluación	Acción
0	0	3	3 == 7 (Falso)	Sigue búsqueda
0	1	1	1 == 7 (Falso)	Sigue búsqueda
1	0	4	4 == 7 (Falso)	Sigue búsqueda
1	1	7	7 == 7 (Verdadero)	matriz[1][1]=99; return;

Diferencia con "Eliminar"

- Sustitución por parámetro:**
A diferencia del método eliminar (que siempre asigna 0), modificar recibe un segundo parámetro dinamizado (nuevoValor) permitiendo actualizar la celda con cualquier número entero.
- Control de flujo optimizado:**
La instrucción return finaliza el método al encontrar la **primera coincidencia**, ahorrando iteraciones en matrices de gran tamaño.

En resumen

1. Búsqueda con for anidado → 2. ¿matriz[i][j] == x? → 3. Asignar nuevoValor y return → 4. Si no, notificar ausencia

3. Método para sumar los elementos de la matriz

```
// 3. Método para sumar los elementos de la matriz
public int sumarElementos() {
    int suma = 0;
    for (int i = 0; i < filas; i++) {
        for (int j = 0; j < columnas; j++) {
            suma += matriz[i][j];
        }
    }
    return suma;
}
```

MÉTODO MATRIZ 3

 **Sumar Elementos de la Matriz**

Recorre la totalidad de las celdas acumulando sus valores para retornar la suma total.



 **¿Qué hace?**

Declara una variable acumuladora (int suma = 0), visita sistemáticamente cada elemento de la matriz con dos ciclos for anidados y va sumando cada valor (suma += matriz[i][j]). Finalmente retorna el valor total entero.

 **Idea principal**

"Utilizamos un acumulador para ir sumando cada valor individual durante el recorrido de la matriz."

 **Ejemplo**

Matriz de entrada (2x3):

315

472

→

SUMA TOTAL
22

3 + 1 + 5 + 4 + 7 + 2 = 22

 **Paso a paso del algoritmo**

1 Inicializar Acumulador

Declara la variable local suma = 0 para almacenar de forma incremental el total de todos los elementos.

```
public int sumarElementos() {
    int suma = 0;
    ...
}
```

Variable neutra en 0

2 Recorrer y Acumular

Los bucles iteran sobre cada celda [i][j], sumando su contenido al acumulador mediante el operador +=.

```
for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        suma += matriz[i][j];
    }
}
```

+ Operador suma equivalente:
suma = suma + matriz[i][j]

3 Retornar el Resultado

Una vez concluidos ambos bucles, se devuelve el valor total acumulado mediante la sentencia return suma;.

```
...
return suma;
}
```

Retorno int directo

 **Prueba de Escritorio (Suma de la Matriz)**

Fila (i)	Col (j)	matriz[i][j]	Operación	Nuevo valor de suma
0	0	3	0 + 3	3
0	1	1	3 + 1	4
0	2	5	4 + 5	9
1	0	4	9 + 4	13
1	1	7	13 + 7	20
1	2	2	20 + 2	22 (Final)

 **Detalle de Firma y Tipo de Retorno**

• **Método no void (Devuelve un valor):**
A diferencia de modificar o eliminar (que son void porque imprimen directamente o cambian estados), este método se declara como public int, obligando el uso de return.

• **Complejidad Temporal:**
Recorre obligatoriamente el 100% de las posiciones de la matriz. Su complejidad es O(filas × columnas) o O(N × M).

 **En resumen**

1. Inicializa suma = 0 → 2. Recorre con for anidado → 3. Acumula suma += matriz[i][j] → 4. return suma

4. Método para hallar el número menor, el número mayor y el promedio

```
// 4. Método para hallar el número menor, el número mayor y el promedio
public void hallarMenorMayorPromedio() {
    if (filas == 0 || columnas == 0) return;

    int menor = matriz[0][0];
    int mayor = matriz[0][0];
    double suma = 0;
    int totalElementos = filas * columnas;

    for (int i = 0; i < filas; i++) {
        for (int j = 0; j < columnas; j++) {
            int valorActual = matriz[i][j];
            if (valorActual < menor) menor = valorActual;
            if (valorActual > mayor) mayor = valorActual;
            suma += valorActual;
        }
    }

    double promedio = suma / totalElementos;

    System.out.println("Número Menor: " + menor);
    System.out.println("Número Mayor: " + mayor);
    System.out.println("Promedio: " + promedio);
}
```



MÉTODO MATRIZ 4

Hallar Menor, Mayor y Promedio

Realiza un único recorrido matricial para calcular valores extremos y la media aritmética.





¿Qué hace?

Inicializa los comparadores menor y mayor con el primer elemento de la matriz (matriz[0][0]). Luego, en un único ciclo anidado, compara y actualiza extremos, suma todos los elementos y calcula el promedio final dividiendo entre el total de celdas.



Idea principal

“Optimizamos en una sola pasada: comparamos mínimo y máximo acumulando la suma total simultáneamente.”



</> Ejemplo

Matriz de entrada (2x3 = 6 elementos):

8	2	5
4	15	2

→

Menor: 2

Mayor: 15

Promedio: 6.0

Suma total = 36 | Elementos = 6 | 36 / 6 = 6.0

Paso a paso del algoritmo

1 Validar e Inicializar

Valida que la matriz no esté vacía. Asigna matriz[0][0] a menor y mayor, y calcula el total de elementos.

```
if (filas == 0 || columnas == 0) return;
int menor = matriz[0][0];
int mayor = matriz[0][0];
double suma = 0;
int totalElementos = filas * columnas;
```

Guard Clause + Base Inicial

2 Recorrer y Comparar

Itera por todas las celdas actualizando menor si el valor es más pequeño, mayor si es más grande, y acumula la suma.

```
for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        int val = matriz[i][j];
        if (val < menor) menor = val;
        if (val > mayor) mayor = val;
        suma += val;
    }
}
```

Evaluación triple simultánea

3 Promedio e Impresión

Divide suma / totalElementos para obtener el promedio decimal de tipo double y muestra las tres estadísticas.

```
double promedio = suma / totalElementos;
System.out.println("Número Menor: " + menor);
System.out.println("Número Mayor: " + mayor);
System.out.println("Promedio: " + promedio);
```

Salida por consola

Prueba de Escritorio (Matriz de Ejemplo)

[i][j]	Valor	¿Menor? (< menor)	¿Mayor? (> mayor)	Suma Acumulada
[0][0]	8	menor = 8	mayor = 8	8
[0][1]	2	Si → menor = 2	No	10
[0][2]	5	No	No	15
[1][0]	4	No	No	19
[1][1]	15	No	Si → mayor = 15	34
[1][2]	2	No (2 < 2 es F)	No	36

Puntos Clave de Programación

- Manejo de Tipos (int vs double): La variable suma se declara como double para que al realizar la división suma / totalElementos, Java efectúe una ****división flotante**** precisa sin perder los decimales.
- Elección del Valor Inicial: Inicializar con matriz[0][0] evita errores comunes como usar 0 (que fallaría si la matriz contiene solo números negativos) o Integer.MAX_VALUE.

✓ En resumen

1. Guard Clause & init con [0][0] → 2. Bucle: actualizar min/máx & suma → 3. promedio = suma / total → 4. Imprimir resultados

5. Método para visualizar la suma de los contenidos que forman la letra L

```
// 5. Método para visualizar la suma de los contenidos que forman la letra L
public int sumarLetraL() {
    int suma = 0;
    System.out.print("Elementos que forman la letra L: ");

    // Primera columna (vertical de la L)
    for (int i = 0; i < filas; i++) {
        suma += matriz[i][0];
        System.out.print(matriz[i][0] + " ");
    }

    // Última fila (horizontal de la L, excluyendo la esquina para no duplicar)
    for (int j = 1; j < columnas; j++) {
        suma += matriz[filas - 1][j];
        System.out.print(matriz[filas - 1][j] + " ");
    }

    System.out.println("\nSuma total de la letra L: " + suma);
    return suma;
}
```

MÉTODO MATRIZ 5

Sumar Elementos de la "Letra L"

L

Suma y muestra los valores de la primera columna (tramo vertical) y la última fila (tramo horizontal).

¿Qué hace?

Recorre en dos etapas no anidadas los bordes que forman la figura de una letra L: primero la primera columna completa (columna 0), y luego el resto de la última fila (filas - 1) desde la segunda posición (j = 1) para evitar sumar dos veces la esquina inferior izquierda.

Idea principal

"Dividimos el trazo en 2 bucles independientes; el segundo inicia en j = 1 para no duplicar la esquina."

</> Ejemplo

Matriz 3x3 (resaltada la figura L):

5	2	9
3	8	1
4	7	6

SUMA "L"
25

Vertical: [5, 3, 4] | Horizontal: [7, 6] → Total = 25

Paso a paso del algoritmo

1 Tramo Vertical (Columna 0)

Un bucle recorre todas las filas de la primera columna ([i][0]), sumando e imprimiendo sus elementos (incluye la esquina inferior).

```
for (int i = 0; i < filas; i++) {
    suma += matriz[i][0];
    System.out.print(matriz[i][0] + " ");
}
```

Recorre de arriba a abajo

2 Tramo Horizontal (Última Fila)

Un segundo bucle recorre la última fila ([filas-1][j]) **empezando desde j = 1** para omitir la esquina ya leída.

```
for (int j = 1; j < columnas; j++) {
    suma += matriz[filas - 1][j];
    System.out.print(matriz[filas - 1][j] + " ");
}
```

Recorre de izquierda a derecha

3 Imprimir y Retornar

Imprime el valor final en una nueva línea indicando la suma total acumulada de la L y retorna el entero suma.

```
System.out.println("\nSuma total de la letra L: " + suma);
return suma;
```

Muestra y devuelve el resultado

Trazo y Prueba de Escritorio (Matriz 3x3)

Tramo	Celda [i][j]	Valor	Operación	Suma Parcial
Vertical (Col 0)	[0][0]	5	0 + 5	5
	[1][0]	3	5 + 3	8
	[2][0] (Esquina)	4	8 + 4	12
Horizontal (Fila 2)	[2][1] (j=1)	7	12 + 7	19
	[2][2] (j=2)	6	19 + 6	25 (Final)

Detalle Crítico del Algoritmo

• Prevención de la "Doble Contabilidad": El elemento inferior izquierdo (en [filas-1][0]) pertenece tanto a la primera columna como a la última fila.

Si el segundo bucle iniciara en j = 0, este valor se sumaría dos veces. Al configurar int j = 1, se garantiza que la intersección se contabilice únicamente una vez en el tramo horizontal.

• Bucles Secuenciales vs Anidados: No usamos bucles anidados porque no estamos recorriendo toda la superficie, sino líneas trazadas en la matriz. Complejidad: O(N + M).

En resumen

1. For 1: Suma Columna 0 (i=0 a filas-1) → 2. For 2: Suma Última Fila (j=1 a col-1) → 3. Retorna suma total de la L

6. Método para visualizar la suma de los contenidos que forman la letra J

```
// 6. Método para visualizar la suma de los contenidos que forman la letra J
public int sumarLetraJ() {
    int suma = 0;
    int colCentro = columnas / 2;
    System.out.print("Elementos que forman la letra J: ");

    // Primera fila completa (línea superior)
    for (int j = 0; j < columnas; j++) {
        suma += matriz[0][j];
        System.out.print(matriz[0][j] + " ");
    }

    // Columna central desde la segunda fila (tallo)
    for (int i = 1; i < filas; i++) {
        suma += matriz[i][colCentro];
        System.out.print(matriz[i][colCentro] + " ");
    }

    // Gancho inferior (última fila a la izquierda de la columna central)
    for (int j = 0; j < colCentro; j++) {
        suma += matriz[filas - 1][j];
        System.out.print(matriz[filas - 1][j] + " ");
    }

    System.out.println("\nSuma total de la letra J: " + suma);
    return suma;
}
```



MÉTODO MATRIZ 6

Sumar Elementos de la "Letra J"

Construye el trazo de la letra J mediante la barra superior, la columna central y el gancho inferior.

J

 ¿Qué hace?

Calcula el eje medio con `colCentro = columnas / 2` y divide la figura en 3 trazos continuos: la barra superior horizontal completa (fila 0), el tallo vertical por el centro (desde fila 1), y el gancho inferior izquierdo en la última fila (antes de llegar al centro).

 Idea clave de intersecciones

"Tallo inicia en `i = 1` y el gancho termina en `j < colCentro` para jamás solapar ningún elemento."

`</>` Ejemplo (Matriz 3x3)

colCentro = 3 / 2 = 1 (Columna 1)

5	2	9
3	8	1
4	7	6

→ SUMA "J" 35

Barra: [5,2,9] + Tallo: [8,7] + Gancho: [4] → Total = 35

Los 3 Trazos del Algoritmo

1 Línea Superior (Barra J)

Recorre toda la primera fila (fila 0) de izquierda a derecha de `j = 0` hasta `columnas - 1`.

```
for (int j = 0; j < columnas; j++) {
    suma += matriz[0][j];
    System.out.print(matriz[0][j] + " ");
}
```

Cubre toda la Fila 0

2 Columna Central (Tallo)

Avanza verticalmente por `colCentro`. Inicia en `i = 1` para **no duplicar** el elemento cruzado de la barra superior.

```
for (int i = 1; i < filas; i++) {
    suma += matriz[i][colCentro];
    System.out.print(matriz[i][colCentro] + " ");
}
```

De la fila 1 hasta el fondo

3 Gancho Inferior Izquierdo

Recorre la última fila (`filas - 1`) desde `j = 0` hasta antes del centro (`j < colCentro`) completando el gancho.

```
for (int j = 0; j < colCentro; j++) {
    suma += matriz[filas - 1][j];
    System.out.print(matriz[filas - 1][j] + " ");
}
```

Se detiene justo antes de `colCentro`

Prueba de Escritorio (Matriz 3x3)

Trazo	Celda [i][j]	Valor	Operación	Suma
1. Barra Sup. (Fila 0)	[0][0]	5	0 + 5	5
	[0][1]	2	5 + 2	7
	[0][2]	9	7 + 9	16
2. Tallo (Col 1)	[1][1] (i=1)	8	16 + 8	24
	[2][1] (i=2)	7	24 + 7	31
3. Gancho	[2][0] (j=0)	4	31 + 4	35 (Final)

Puntos Clave del Algoritmo

- Cálculo del Centro: `int colCentro = columnas / 2;` realiza una división entera. En una matriz de 3 columnas da 1; en 5 columnas da 2.
- Control Estricto de Intersecciones:
 1. Cruce Superior: El tallo inicia en `i = 1` para no repetir la celda [0][colCentro] leída en la Barra.
 2. Cruce Inferior: El gancho corre mientras `j < colCentro`, deteniéndose justo en la columna anterior para no duplicar la base del tallo [filas-1][colCentro].

En resumen

1. For 1: Barra Fila 0 Completa → 2. For 2: Tallo ColCentro (i=1..filas-1) → 3. For 3: Gancho Fila Base (j=0..colCentro-1)

7. Método para visualizar la suma de los contenidos que forman la letra U

```
// 7. Método para visualizar la suma de los contenidos que forman la letra U
public int sumarLetraU() {
    int suma = 0;
    System.out.print("Elementos que forman la letra U: ");

    // Columna izquierda completa
    for (int i = 0; i < filas; i++) {
        suma += matriz[i][0];
        System.out.print(matriz[i][0] + " ");
    }

    // Base inferior (última fila sin las esquinas)
    for (int j = 1; j < columnas - 1; j++) {
        suma += matriz[filas - 1][j];
        System.out.print(matriz[filas - 1][j] + " ");
    }

    // Columna derecha completa
    for (int i = 0; i < filas; i++) {
        suma += matriz[i][columnas - 1];
        System.out.print(matriz[i][columnas - 1] + " ");
    }

    System.out.println("\nSuma total de la letra U: " + suma);
    return suma;
}
```



MÉTODO MATRIZ 7

Sumar Elementos de la "Letra U"

Recorre la pared izquierda completa, la base inferior (sin esquinas) y la pared derecha completa.

U

¿Qué hace?

Recorre el contorno inferior en forma de U dividiéndolo en 3 tramos limpios: 1. La columna izquierda completa (col 0). 2. La base inferior (filas - 1) excluyendo ambas esquinas para no duplicar. 3. La columna derecha completa (columnas - 1).

Exclusión Doble de Esquinas

"La base empieza en j = 1 y termina en j < columnas - 1 para dejar ambas esquinas a sus columnas."

</> Ejemplo (Matriz 3x3)

Matriz 3x3 (resaltado trazo U):

5	2	9
3	8	1
4	7	6

SUMA "U"
35

Izquierda: [5,3,4] + Base: [7] + Derecha: [9,1,6] → Total = 35

Los 3 Trazos del Algoritmo

1 Columna Izquierda

Bucle vertical que recorre la primera columna ([i][0]) desde i = 0 hasta filas - 1 (incluye la esquina inferior izq.).

```
for (int i = 0; i < filas; i++) {
    suma += matriz[i][0];
    System.out.print(matriz[i][0] + " ");
}
```

De arriba a abajo (Columna 0)

2 Base Inferior (Sin Esquinas)

Recorre la última fila (filas - 1) desde j = 1 hasta columnas - 2, omitiendo ambas esquinas.

```
for (int j = 1; j < columnas - 1; j++) {
    suma += matriz[filas - 1][j];
    System.out.print(matriz[filas - 1][j] + " ");
}
```

De j=1 hasta j < columnas - 1

3 Columna Derecha

Bucle vertical que recorre la última columna ([i][columnas - 1]) completa (incluye la esquina inferior der.).

```
for (int i = 0; i < filas; i++) {
    suma += matriz[i][columnas - 1];
    System.out.print(matriz[i][columnas - 1] + " ");
}
```

De arriba a abajo (Última Columna)

Prueba de Escritorio (Matriz 3x3)

Trazo	Celda [i][j]	Valor	Operación	Suma
1. Col Izq. (Col 0)	[0][0]	5	0 + 5	5
	[1][0]	3	5 + 3	8
	[2][0] (Esq. Izq.)	4	8 + 4	12
2. Base	[2][1] (j=1)	7	12 + 7	19
3. Col Der. (Col 2)	[0][2]	9	19 + 9	28
	[1][2]	1	28 + 1	29
	[2][2] (Esq. Der.)	6	29 + 6	35 (Final)

Conceptos Claves de Intersección

¿Por qué la base excluye ambos extremos?

La esquina inferior izquierda [filas-1][0] es procesada por el primer bucle (columna izquierda).

La esquina inferior derecha [filas-1][columnas-1] es procesada por el tercer bucle (columna derecha).

Condición del bucle central:

Al definir int j = 1 y la condición j < columnas - 1, la base solo toma los elementos intermedios estrictamente internos de la fila base.

Eficiencia del trazo:

Se recorren los elementos exactos del perímetro inferior con complejidad lineal O(N).

En resumen

1. For 1: Suma Columna Izquierda (0) → 2. For 2: Suma Base Central (j=1..col-2) → 3. For 3: Suma Columna Derecha (col-1)

8. Método para visualizar el menor número de cada fila
9. Método para contar los números pares en la matriz

```
// 8. Método para visualizar el menor número de cada fila
public void mostrarMenorPorFila() {
    System.out.println("Menor número de cada fila:");
    for (int i = 0; i < filas; i++) {
        int menorFila = matriz[i][0];
        for (int j = 1; j < columnas; j++) {
            if (matriz[i][j] < menorFila) {
                menorFila = matriz[i][j];
            }
        }
        System.out.println("Fila " + i + ": " + menorFila);
    }
}

// 9. Método para contar los números pares en la matriz
public int contarPares() {
    int contador = 0;
    for (int i = 0; i < filas; i++) {
        for (int j = 0; j < columnas; j++) {
            if (matriz[i][j] % 2 == 0) {
                contador++;
            }
        }
    }
    System.out.println("Cantidad de números pares en la matriz: " + contador);
    return contador;
}
```



MÉTODOS MATRIZ 8 & 9

Búsqueda de Mínimos y Contabilidad Par

Múltiples recorridos: Mínimo por fila aislada vs. Conteo global condicional con módulo (%).



Método 8: mostrarMenorPorFila()

Retorno: void (Imprime directo)

```
public void mostrarMenorPorFila() {
    System.out.println("Menor número de cada fila:");
    for (int i = 0; i < filas; i++) {
        int menorFila = matriz[i][0]; // Reinicia por fila
        for (int j = 1; j < columnas; j++) {
            if (matriz[i][j] < menorFila) {
                menorFila = matriz[i][j]; // Actualiza mínimo
            }
        }
        System.out.println("Fila " + i + ": " + menorFila);
    }
}
```

1. Inicialización local

menorFila = matriz[i][0] asume que el primer elemento de la fila actual es el menor preliminar.

2. Inicio en j = 1

El bucle interno empieza en la segunda columna (j = 1) para evitar comparar el elemento cero consigo mismo.

3. Reinicio automático

Al estar declarada DENTRO del primer for, la variable menorFila se reinicia para cada nueva fila i.

Método 9: contarPares()

Retorno: int (Devuelve total)

```
public int contarPares() {
    int contador = 0; // Acumulador global
    for (int i = 0; i < filas; i++) {
        for (int j = 0; j < columnas; j++) {
            if (matriz[i][j] % 2 == 0) {
                contador++; // Incrementa si es par
            }
        }
    }
    System.out.println("\nCantidad de pares: " + contador);
    return contador;
}
```

Conteo de Pares:

Matriz 3x3

Fila 0: 8 2 5 2 pares

Fila 1: 9 7 4 1 par

Fila 2: 1 6 3 1 par

Total Pares Encontrados: 4

Evaluación Paridad:

matriz[i][j] % 2 == 0

El operador % (módulo) calcula el residuo de la división por 2. Si es 0, es Par.

Ámbito Variable: Mínimo por Fila

La variable menorFila NACE y MUERE dentro del bucle de 1. Se resetea de forma aislada para cada fila individual de la matriz.

Ámbito Variable: Acumulador Global

La variable contador se declara ANTES de los dos bucles. Sobrevive a toda la iteración para acumular los pares totales de la matriz.

1 Método 8: Recorre filas, evalúa mínimo local y lo imprime por cada fila.

2 Método 9: Recorre todo (i,j), evalúa % 2 == 0 y retorna el conteo.

10. Método para mover los contenidos de las filas una posición a la derecha public void rotarFilasDerecha() {



MÉTODOS MATRIZ 10 & AUXILIAR

Rotación Horizontal de Filas y Visualización

Desplazamiento circular hacia la derecha (Shift Right) e impresión limpia con Arrays.toString().



Método 10: rotarFilasDerecha()

Rotación Circular por Fila

```
public void rotarFilasDerecha() {
    for (int i = 0; i < filas; i++) {
        // 1. Resguardar el último elemento de la fila
        int ultimo = matriz[i][columnas - 1];

        // 2. Desplazamiento hacia la derecha (de atrás a adelante)
        for (int j = columnas - 1; j > 0; j--) {
            matriz[i][j] = matriz[i][j - 1];
        }

        // 3. El último pasa a ser el primero
        matriz[i][0] = ultimo;
    }
    System.out.println("Se movieron los elementos de cada fila...");
}
```

Secuencia en Fila [i]:

Ejemplo: [1, 2, 3]

Original: 1 2 3 ultimo = 3

Shift →: 1 1 2 Desplaza der.

Final: 3 1 2 matriz[i][0]=3

PASO 1

Resguardo Temporal

Guarda matriz[i][columnas - 1] en la variable ultimo antes de que sea sobrescrito por el desplazamiento.

PASO 2

Bucle Inverso (j--)

Recorre desde columnas - 1 descendiendo hasta j > 0 para copiar cada elemento previo [j - 1] a la derecha.

PASO 3

Reubicación Final

Asigna matriz[i][0] = ultimo. Así el elemento expulsado por la derecha reingresa por la izquierda.

⚠ Atención! ¿Por qué el bucle de j va hacia atrás (j--)?

Si recorriéramos de izquierda a derecha (j = 0; j < cols; j++), el valor matriz[i][0] sobrescribiría a matriz[i][1], y en el siguiente paso ese mismo valor sobrescribiría a matriz[i][2], duplicando el primer elemento en toda la fila. Al ir hacia atrás (j--), no perdemos información.

Método Auxiliar: mostrarMatriz()

Impresión Limpia por Consola

```
public void mostrarMatriz() {
    System.out.println("Matriz actual:");
    for (int[] fila : matriz) {
        System.out.println(Arrays.toString(fila));
    }
}
```

📄 For-Each sobre Matriz Bidimensional

for (int[] fila : matriz) extrae cada fila completa de la matriz como un arreglo unidimensional (int[]).

🔧 Utility Class: Arrays.toString()

Convierte la fila entera en un formato de texto legible [elem1, elem2, elem3] evitando bucles internos adicionales.

🔄 Rotación Horizontal: Guarda el último → Desplaza con j-- → Coloca al inicio.

🖨 Impresión Java: Arrays.toString(fila) formatea automáticamente cada fila.